Parallel Computing (BCS702) - Lab Programs

Program 1

OpenMP sequential vs parallel merge sort (sections)

Code Copy

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
void merge(int a[],int l,int m,int r){
 int i=l,j=m+1,k=0,t[r-l+1];
 while(i<=m&&j<=r)t[k++]=a[i]<=a[j]?a[i++]:a[j++];
 while(i<=m)t[k++]=a[i++];
 while(j<=r)t[k++]=a[j++];
 for(k=0;l<=r;)a[l++]=t[k++];
}
void sort(int a[],int l,int r,int p){
 if(l>=r)return;
 int m=(l+r)/2;
 #pragma omp parallel sections if(p)
 {
 #pragma omp section
 sort(a,l,m,p);
 #pragma omp section
 sort(a,m+1,r,p);
 }
 merge(a,l,m,r);
}
int main(){
 int n=100000,a[n],b[n];
 double s;
 for(int i=0;i<n;i++)a[i]=b[i]=rand()%1000;
 s=omp_get_wtime();
 sort(a,0,n-1,0);
 printf("Sequential Merge Sort Time: %f seconds\n",omp_get_wtime()-s);
 s=omp_get_wtime();
 sort(b,0,n-1,1);
 printf("Parallel Merge Sort Time: %f seconds\n",omp_get_wtime()-s);
}
```

Compile & Run Copy

```
Compile: gcc -fopenmp prg1.c -o prg1
Run: export OMP_NUM_THREADS=4 && ./prg1
```

Program 2

OpenMP schedule(static,2) iteration to thread mapping

Code Copy

```
#include <stdio.h>
#include <omp.h>
int main(){
 int n;
 printf("Enter number of iterations: ");
 scanf("%d",&n);
 int t[n];
 #pragma omp parallel for schedule(static,2)
 for(int i=0;i<n;i++)t[i]=omp_get_thread_num();
 for(int i=0;i<n;i+=2)
 printf("Thread %d : Iterations %d -- %d\n",t[i],i,i+1<n?i+1:i);
}
```

Compile & Run Copy

```
Compile: gcc -fopenmp prg2.c -o prg2
Run: export OMP_NUM_THREADS=4 && ./prg2
```

Program 3

OpenMP Fibonacci using tasks

Code Copy

```
#include <stdio.h>
#include <omp.h>
int fib(int n){
 if(n<2)return n;
 int x,y;
 #pragma omp task shared(x)
 x=fib(n-1);
 y=fib(n-2);
 #pragma omp taskwait
 return x+y;
}
int main(){
 int n;
 printf("Enter number of Fibonacci terms: ");
 scanf("%d",&n);
 printf("Fibonacci Series:\n");
 #pragma omp parallel
 #pragma omp single
```

```
 for(int i=0;i<n;i++)printf("%d ",fib(i));
 printf("\n");
}
```

Compile & Run Copy

```
Compile: gcc -fopenmp prg3.c -o prg3
Run: export OMP_NUM_THREADS=4 && ./prg3
```

Program 4

OpenMP primes with parallel for (serial vs parallel time)

Code Copy

```
#include <stdio.h>
#include <omp.h>
int prime(int n){
 if(n<2)return 0;
 for(int i=2;i*i<=n;i++)
 if(n%i==0)return 0;
 return 1;
}
int main(){
 int n,c=0;
 double s;
 printf("Enter the value of n: ");
 scanf("%d",&n);
 printf("\nPrime numbers from 1 to %d:\n",n);
 for(int i=1;i<=n;i++)
 if(prime(i))printf("%d ",i);
 printf("\n");
 s=omp_get_wtime();
 for(int i=1;i<=n;i++)c+=prime(i);
 printf("Serial Time: %f seconds\n",omp_get_wtime()-s);
 c=0;
 s=omp_get_wtime();
 #pragma omp parallel for reduction(+:c)
 for(int i=1;i<=n;i++)c+=prime(i);
 printf("Parallel Time: %f seconds\n",omp_get_wtime()-s);
}
```

Compile & Run Copy

```
Compile: gcc -fopenmp prg4.c -o prg4
Run: export OMP_NUM_THREADS=4 && ./prg4
```

Program 5

MPI_Send and MPI_Recv

Code Copy

```
#include <stdio.h>
#include <mpi.h>
int main(){
 int rank,n=100;
 MPI_Init(NULL,NULL);
 MPI_Comm_rank(MPI_COMM_WORLD,&rank);
 if(rank==0){
 printf("Process 0 sending number %d to process 1\n",n);
 MPI_Send(&n,1,MPI_INT,1,0,MPI_COMM_WORLD);
 }
 else if(rank==1){
 MPI_Recv(&n,1,MPI_INT,0,0,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
 printf("Process 1 received number %d from process 0\n",n);
 }
 MPI_Finalize();
}
```

Compile & Run Copy

```
Compile: mpicc prg5.c -o prg5
Run: mpirun -np 2 ./prg5
```

Program 6

MPI deadlock and its avoidance (MODE 1 = deadlock, MODE 2 = fixed)

Code Copy

```
#include <stdio.h>
#include <mpi.h>
#define MODE 2
int main(){
 int rank,n=123;
 MPI_Init(NULL,NULL);
 MPI_Comm_rank(MPI_COMM_WORLD,&rank);
 if(MODE==1){
 printf("Process %d waiting to receive from Process %d...\n",rank,1-rank);
 MPI_Recv(&n,1,MPI_INT,1-rank,0,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
 MPI_Send(&n,1,MPI_INT,1-rank,0,MPI_COMM_WORLD);
 }
 else if(rank==0){
 MPI_Send(&n,1,MPI_INT,1,0,MPI_COMM_WORLD);
 MPI_Recv(&n,1,MPI_INT,1,0,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
 printf("Process 0 received back number: %d\n",n);
```

```
 printf("Process 0 received back number: %d\n",n);
 }
 else{
 MPI_Recv(&n,1,MPI_INT,0,0,MPI_COMM_WORLD,MPI_STATUS_IGNORE);
 MPI_Send(&n,1,MPI_INT,0,0,MPI_COMM_WORLD);
 printf("Process 1 received and sent back number: %d\n",n);
 }
 MPI_Finalize();
}
```

Compile & Run Copy

```
Compile: mpicc prg6.c -o prg6
Run: mpirun -np 2 ./prg6
```

Program 7

MPI_Bcast

Code Copy

```
#include <stdio.h>
#include <mpi.h>
int main(){
 int rank,n;
 MPI_Init(NULL,NULL);
 MPI_Comm_rank(MPI_COMM_WORLD,&rank);
 if(rank==0){
 n=50;
 printf("Process 0 broadcasting number 50 to all other processes.\n");
 }
 MPI_Bcast(&n,1,MPI_INT,0,MPI_COMM_WORLD);
 printf("Process %d received number %d\n",rank,n);
 MPI_Finalize();
}
```

Compile & Run Copy

```
Compile: mpicc prg7.c -o prg7
Run: mpirun -np 4 ./prg7
```

Program 8

MPI_Scatter and MPI_Gather

Code Copy

```
#include <stdio.h>
#include <mpi.h>
int main(){
 int rank,size,x,a[100],g[100];
 MPI_Init(NULL,NULL);
 MPI_Comm_rank(MPI_COMM_WORLD,&rank);
 MPI_Comm_size(MPI_COMM_WORLD,&size);
 if(rank==0)
 for(int i=0;i<size;i++)a[i]=i*10;
 MPI_Scatter(a,1,MPI_INT,&x,1,MPI_INT,0,MPI_COMM_WORLD);
 x+=rank;
 MPI_Gather(&x,1,MPI_INT,g,1,MPI_INT,0,MPI_COMM_WORLD);
 if(rank==0){
 printf("Gathered data in root process:\n");
 for(int i=0;i<size;i++)
 printf("gathered_data[%d] = %d\n",i,g[i]);
 }
 MPI_Finalize();
}
```

Compile & Run Copy

```
Compile: mpicc prg8.c -o prg8
Run: mpirun -np 4 ./prg8
```

Program 9

MPI_Reduce and MPI_Allreduce

Code Copy

```
#include <stdio.h>
#include <mpi.h>
int main(){
 int rank,v,r[4],a[4];
 MPI_Op op[4]={MPI_SUM,MPI_PROD,MPI_MAX,MPI_MIN};
 MPI_Init(NULL,NULL);
 MPI_Comm_rank(MPI_COMM_WORLD,&rank);
 v=rank+1;
 for(int i=0;i<4;i++){
 MPI_Reduce(&v,&r[i],1,MPI_INT,op[i],0,MPI_COMM_WORLD);
 MPI_Allreduce(&v,&a[i],1,MPI_INT,op[i],MPI_COMM_WORLD);
 }
 if(rank==0){
 printf("--- Results using MPI_Reduce (only root prints) ---\n");
 printf("Sum = %d\nProduct = %d\nMax = %d\nMin = %d\n",r[0],r[1],r[2],r[3]);
 }
 printf("Process %d has value %d\n",rank,v);
 printf("Process %d sees (MPI_Allreduce): Sum=%d, Prod=%d, Max=%d, Min=%d\n",
 rank,a[0],a[1],a[2],a[3]);
```

```
 rank,a[0],a[1],a[2],a[3]);
 MPI_Finalize();
}
```

Compile & Run Copy

```
Compile: mpicc prg9.c -o prg9
Run: mpirun -np 4 ./prg9
```
